
Especificación de Requisitos de Software

para

Misión Oasis Lunar (Simulador Artemis II)

Versión 1.0 aprobada

Preparado por

CDR: Laura Priscila Guerrero Echavarría

REQ: Yanantonys Germosen López

ARCH: Patrick Schrils Pérez

FDO: Rommel Arturo Santana Arias

CAPCOM: Martín Isaac Jiménez Gabot

Mision Oasis Lunar

7/6/2026

Tabla de contenido

Índice de contenidos	ii
Historial de revisiones	ii
1. Introducción	1
1.1 Propósito	1
1.2 Convenciones del documento	1
1.3 Público objetivo y sugerencias de lectura	1
1.4 Alcance del producto	1
1.5 Referencias	1
2. Descripción general	2
2.1 Perspectiva del producto	2
2.2 Funciones del producto	2
2.3 Clases de usuario y características	2
2.4 Entorno operativo	2
2.5 Restricciones de diseño e implementación	2
2.6 Documentación del usuario	2
2.7 Supuestos y dependencias	3
3. Requisitos de interfaz externa	3
3.1 Interfaces de usuario	3
3.2 Interfaces de hardware	3
3.3 Interfaces de software	3
3.4 Interfaces de comunicaciones	3
4. Características del sistema	4
4.1 Característica del sistema 1	4
4.2 Característica del sistema 2 (y así sucesivamente)	4
5. Otros requisitos no funcionales	4
5.1 Requisitos de desempeño	4
5.2 Requisitos de seguridad	5
5.3 Requisitos de seguridad	5
5.4 Atributos de calidad del software	5
5.5 Reglas de negocio	5
6. Otros requisitos	5
Apéndice A: Glosario	5
Apéndice B: Modelos de análisis	5
Apéndice C: Lista por determinar	6

Historial de revisiones

Nombre	Fecha	Motivo de los cambios	Versión

1. Introducción

1.1 Objetivo

*Este documento corresponde a la Especificación de Requisitos de Software (Software Requirements Specification – SRS) del proyecto **Misión Oasis Lunar**, versión 1.0. El propósito de este documento es definir de manera formal, clara y verificable los requisitos funcionales y no funcionales del sistema, proporcionando una referencia común para todos los participantes involucrados en el desarrollo, validación, mantenimiento y evaluación del software.*

Misión Oasis Lunar es un simulador aeroespacial interactivo inspirado en la misión Artemis II de la NASA. El sistema permitirá modelar y visualizar trayectorias orbitales translunares mediante el uso de la biblioteca Orekit para los cálculos de dinámica espacial y JavaFX para la representación gráfica e interacción con el usuario.

El alcance de este documento comprende la totalidad del sistema que será desarrollado durante el Proyecto Simulador de Misión Lunar. Se incluyen los requisitos relacionados con la propagación orbital, visualización gráfica, telemetría, control de simulación, gestión temporal y sistema de alertas. Asimismo, se establecen las restricciones técnicas, dependencias, interfaces externas y atributos de calidad que deberán cumplirse durante el desarrollo del producto.

Este SRS servirá como base para las actividades de diseño, implementación, pruebas, validación y mantenimiento del sistema, además de proporcionar criterios objetivos para verificar que el producto desarrollado satisface las necesidades identificadas durante la fase de ingeniería de requisitos.

1.2 Convenciones de documentos

Este documento ha sido elaborado siguiendo la estructura recomendada por el estándar IEEE 830 para la especificación de requisitos de software y tomando como referencia las recomendaciones de la NASA para la redacción de requisitos claros, completos, verificables y consistentes.

Las siguientes convenciones serán utilizadas a lo largo del documento:

- *Los requisitos funcionales se identificarán mediante el prefijo **RF** (Requisito Funcional).*
- *Los requisitos no funcionales se identificarán mediante el prefijo **RNF** (Requisito No Funcional).*
- *Los requisitos detallados utilizarán identificadores únicos con formato jerárquico para facilitar su trazabilidad.*
- *Las secciones pendientes de definición se identificarán mediante la etiqueta **TBD (To Be Determined)**.*
- *Los términos técnicos relevantes aparecerán definidos en el glosario incluido en los apéndices del documento.*

Las prioridades de los requisitos se clasificarán utilizando la siguiente escala:

- **Alta:** funcionalidad indispensable para cumplir los objetivos principales del sistema.
- **Media:** funcionalidad importante que mejora significativamente el producto.
- **Baja:** funcionalidad complementaria cuya ausencia no impide la operación principal del sistema.

Cada requisito conservará su propia prioridad independientemente de la prioridad asignada a la característica o módulo al que pertenezca.

Las referencias a componentes externos, bibliotecas y tecnologías se realizarán utilizando sus nombres oficiales para facilitar la identificación de dependencias y documentación técnica asociada.

1.3 Público objetivo y sugerencias de lectura

Este documento está dirigido a todas las partes interesadas involucradas en el desarrollo y evaluación del proyecto.

Desarrolladores de Software

Utilizarán este documento como referencia principal para implementar las funcionalidades requeridas y verificar el cumplimiento de los requisitos establecidos.

Arquitectos de Software

Utilizarán el documento para diseñar la estructura técnica del sistema, definir la integración entre componentes y garantizar el cumplimiento de las restricciones arquitectónicas.

Ingenieros de Requisitos

Serán responsables de validar la consistencia, trazabilidad y completitud de los requisitos definidos en este SRS.

Profesor Evaluador

Utilizará el documento para verificar que el proyecto cumple con los objetivos académicos y las buenas prácticas de ingeniería de requisitos establecidas para la asignatura.

Integrantes del Equipo del Proyecto

Utilizarán este documento como referencia común para la planificación, implementación y seguimiento de las actividades de desarrollo.

Futuros Mantenedores

Podrán utilizar esta especificación para comprender la funcionalidad del sistema y facilitar futuras modificaciones o ampliaciones.

Organización del Documento

El documento se encuentra organizado de la siguiente manera:

- *La Sección 1 presenta la introducción y el propósito del documento.*
- *La Sección 2 proporciona una descripción general del sistema.*
- *La Sección 3 describe las interfaces externas y la interacción con usuarios y componentes externos.*
- *La Sección 4 especifica las características del sistema y sus requisitos funcionales asociados.*
- *La Sección 5 define los requisitos no funcionales y atributos de calidad.*
- *Los apéndices incluyen glosarios, modelos conceptuales y elementos pendientes de definición.*

Se recomienda que los lectores comiencen por la Introducción y la Descripción General antes de consultar las secciones específicas relacionadas con sus responsabilidades dentro del proyecto.

1.4 Alcance del producto

Misión Oasis Lunar es un simulador orbital educativo diseñado para reproducir aspectos fundamentales de una misión translunar inspirada en Artemis II, una de las misiones tripuladas del programa Artemis de la NASA.

El propósito principal del sistema es proporcionar una herramienta que permita aplicar conocimientos de ingeniería de software, simulación computacional y dinámica orbital dentro de un entorno académico controlado. El software permitirá calcular trayectorias espaciales, representar visualmente el movimiento de una nave, monitorear parámetros de telemetría y gestionar eventos operativos durante la simulación.

Entre los principales beneficios esperados se encuentran:

- *Facilitar la comprensión de conceptos relacionados con mecánica orbital y exploración espacial.*
- *Aplicar metodologías de ingeniería de requisitos y desarrollo de software en un proyecto multidisciplinario.*
- *Proporcionar una plataforma extensible para futuras simulaciones relacionadas con misiones espaciales.*
- *Promover el uso de herramientas profesionales utilizadas en el ámbito aeroespacial, como Orekit.*

El proyecto se alinea con los objetivos académicos de la asignatura de Ingeniería de Requisitos, permitiendo que los estudiantes desarrollen competencias relacionadas con análisis, documentación, diseño y validación de sistemas de software complejos.

1.5 Referencias

Las siguientes fuentes han sido utilizadas como base para la elaboración de este documento y deberán consultarse para obtener información adicional relacionada con los requisitos del sistema:

1. NASA. Artemis II Mission Overview. Programa Artemis. NASA. Versión pública vigente. Disponible en: <https://www.nasa.gov>
2. Orekit Development Team. Orekit Documentation. Biblioteca de dinámica espacial para Java. Versión vigente. Disponible en: <https://www.orekit.org>
3. IEEE Computer Society. IEEE Recommended Practice for Software Requirements Specifications (IEEE 830). IEEE Standards Association.
4. Sommerville, Ian. Software Engineering. Décima edición. Pearson Education.
5. Oracle Corporation. Java Platform, Standard Edition Documentation (Java SE 17). Disponible en: <https://docs.oracle.com>
6. OpenJFX Community. JavaFX Documentation. Disponible en: <https://openjfx.io>
7. NASA. NASA Systems Engineering Handbook. NASA/SP-2016-6105 Rev2.
8. NASA. Appendix C – How to Write a Good Requirement. Disponible en la documentación oficial de ingeniería de sistemas de la NASA.
9. Repositorio oficial del proyecto Misión Oasis Lunar (<https://github.com/PatrickS-P/Mision-Oasis-Lunar---Equipo-3>).
10. Documento de decisiones técnicas del proyecto (decisiones.md), versión vigente mantenida por el equipo de desarrollo.

2. Descripción general

2.1 Perspectiva del producto

Misión Oasis Lunar es un producto de software nuevo e independiente desarrollado con fines académicos para la simulación de una misión espacial inspirada en Artemis II. El sistema tiene como objetivo proporcionar una plataforma interactiva que permita visualizar y analizar aspectos fundamentales de una misión translunar, integrando conceptos de dinámica orbital, simulación temporal y monitoreo de telemetría.

El producto forma parte del Proyecto Simulador de Misión Lunar desarrollado en la asignatura de Ingeniería de Requisitos y constituye la base sobre la cual se implementarán futuras funcionalidades relacionadas con simulaciones espaciales más complejas. Aunque no reemplaza ningún sistema existente, toma como referencia documentación pública de la misión Artemis II y emplea tecnologías utilizadas en aplicaciones aeroespaciales reales.

La arquitectura del sistema está compuesta por cuatro subsistemas principales:

- Motor de Propagación Orbital (Orekit)
- Módulo de Simulación Temporal
- Módulo de Telemetría y Alertas
- Interfaz Gráfica de Usuario (JavaFX)

El flujo general de funcionamiento comienza cuando el usuario inicia una simulación. El motor orbital calcula la posición y velocidad de la nave, los resultados son procesados por el módulo de telemetría y posteriormente enviados a la interfaz gráfica para su visualización. Paralelamente, el sistema supervisa condiciones operativas que puedan generar alertas.

A nivel conceptual, la arquitectura puede representarse de la siguiente forma:

Usuario → Interfaz JavaFX → Controlador de Simulación → Motor Orekit → Datos Orbitales → Telemetría y Alertas → Interfaz Gráfica

Esta estructura desacoplada facilita el mantenimiento, la escalabilidad y la incorporación futura de nuevas funcionalidades sin afectar significativamente los componentes existentes.

2.2 Funciones del producto

El sistema proporcionará las siguientes funciones principales:

- *Simular trayectorias orbitales translunares utilizando modelos físicos implementados mediante Orekit.*
- *Calcular y actualizar continuamente la posición y velocidad de la nave espacial durante la misión.*
- *Representar gráficamente la Tierra, la Luna y la nave espacial dentro de una vista orbital interactiva.*
- *Mostrar información de telemetría en tiempo real, incluyendo velocidad orbital, altitud y tiempo transcurrido de misión.*
- *Permitir al usuario iniciar, pausar y reiniciar la simulación mediante controles gráficos.*
- *Gestionar el avance temporal de la misión y sincronizar los cálculos físicos con la representación visual.*
- *Detectar condiciones operativas relevantes y generar alertas visuales para el usuario.*
- *Registrar eventos importantes producidos durante la simulación para facilitar el análisis posterior.*
- *Mantener la consistencia entre los cálculos orbitales, la telemetría y la representación gráfica.*
- *Facilitar futuras ampliaciones relacionadas con nuevas misiones espaciales y modelos físicos más avanzados.*

Estas funciones se encuentran estrechamente relacionadas. Los cálculos realizados por el motor orbital alimentan los módulos de telemetría y visualización, mientras que el sistema de alertas utiliza los mismos datos para detectar desviaciones o condiciones críticas.

2.3 Clases de usuario y características

El sistema está diseñado para ser utilizado por diferentes tipos de usuarios que participan en el desarrollo y evaluación del proyecto.

Comandante (CDR)

El Comandante representa al usuario principal del sistema y es responsable de controlar la simulación durante su ejecución.

Características:

- *Frecuencia de uso alta.*
- *Utiliza la mayoría de las funcionalidades del sistema.*
- *Requiere conocimientos básicos de la misión simulada.*
- *Tiene acceso a los controles de inicio, pausa y reinicio.*

Prioridad de atención: Alta.

Especialista de Requisitos (REQ)

El Especialista de Requisitos utiliza el sistema para verificar el cumplimiento de los requisitos funcionales y no funcionales definidos en la documentación.

Características:

- *Frecuencia de uso media.*
- *Nivel técnico medio-alto.*
- *Se enfoca en la validación y verificación del comportamiento esperado.*
- *Requiere acceso a información de telemetría y eventos del sistema.*

Prioridad de atención: Alta.

Arquitecto de Software (ARCH)

El Arquitecto supervisa la estructura técnica del proyecto y verifica que la implementación cumpla con las decisiones arquitectónicas definidas por el equipo.

Características:

- *Frecuencia de uso media.*
- *Nivel técnico alto.*
- *Interés principal en la integración entre módulos y el desempeño general del sistema.*
- *Participa en actividades de mantenimiento y evolución del software.*

Prioridad de atención: Media.

CAPCOM

El CAPCOM monitorea la información operativa y las alertas generadas durante la misión simulada.

Características:

- *Frecuencia de uso media.*

- *Nivel técnico medio.*
- *Se enfoca en el seguimiento de eventos y estados críticos.*
- *Requiere acceso constante a la consola de alertas y telemetría.*

Prioridad de atención: Media.

2.4 Entorno operativo

El sistema deberá ejecutarse sobre plataformas de escritorio modernas compatibles con Java.

Hardware recomendado:

- *Procesador Dual Core o superior.*
- *Memoria RAM mínima de 4 GB.*
- *Espacio disponible en disco de al menos 500 MB.*
- *Resolución mínima de pantalla de 1366 × 768 píxeles.*

Sistemas operativos compatibles:

- *Windows 10 o superior.*
- *Linux (Ubuntu 22.04 o equivalente).*
- *macOS 12 o superior.*

Software requerido:

- *Java Development Kit (JDK) 17 o superior.*
- *JavaFX SDK 17 o superior.*
- *Biblioteca Orekit.*
- *Git para gestión de versiones.*

El sistema deberá coexistir adecuadamente con otras aplicaciones ejecutándose simultáneamente en el equipo sin producir conflictos de recursos o bloqueos del sistema operativo.

2.5 Restricciones de diseño e implementación

El desarrollo del proyecto estará sujeto a diversas restricciones técnicas y académicas.

El lenguaje de programación oficial será Java debido a los requisitos establecidos para el proyecto y a la compatibilidad con Orekit y JavaFX.

La biblioteca Orekit deberá utilizarse obligatoriamente para la propagación orbital y cálculos relacionados con dinámica espacial. No se permitirá reemplazar estos cálculos por implementaciones simplificadas sin justificación técnica.

La interfaz gráfica deberá desarrollarse utilizando JavaFX para garantizar compatibilidad con el entorno tecnológico seleccionado.

El control de versiones deberá gestionarse mediante Git y GitHub siguiendo la política de ramificación acordada por el equipo.

La arquitectura deberá implementar principios de programación orientada a objetos, modularidad y separación de responsabilidades.

Los cálculos orbitales deberán ejecutarse en hilos independientes para evitar bloqueos de la interfaz gráfica y garantizar una experiencia de usuario fluida.

El sistema deberá cumplir con las normas académicas y lineamientos definidos para el Proyecto Simulador de Misión Lunar.

2.6 Documentación del usuario

La documentación entregable deberá permitir la correcta instalación, utilización y mantenimiento del sistema.

Los documentos previstos incluyen:

- *README.md del repositorio.*
- *Manual de instalación.*
- *Manual de usuario.*
- *Documento de Especificación de Requisitos de Software (SRS).*
- *Registro de riesgos del proyecto.*
- *Archivo decisiones.md.*
- *Documentación técnica del código fuente.*
- *Guías de configuración del entorno de desarrollo.*

Toda la documentación deberá entregarse en formato digital mediante el repositorio GitHub del proyecto y utilizar formatos estándar como Markdown, PDF o DOCX.

2.7 Supuestos y dependencias

El proyecto se basa en varios supuestos que podrían afectar el desarrollo y funcionamiento del sistema.

Se asume que todos los integrantes dispondrán de acceso a un entorno compatible con Java 17 y las bibliotecas requeridas.

Se asume que Orekit proporcionará las capacidades necesarias para implementar los cálculos orbitales previstos en el alcance del proyecto.

Se asume que la documentación pública de Artemis II contiene información suficiente para definir escenarios de simulación representativos.

Se asume que GitHub permanecerá disponible para la gestión del código fuente y la colaboración entre los miembros del equipo.

Las principales dependencias externas del sistema incluyen:

- *Biblioteca Orekit.*
- *JavaFX SDK.*
- *Java Development Kit (JDK).*
- *Git y GitHub.*
- *Máquina Virtual de Java (JVM).*

Cambios significativos en cualquiera de estas dependencias podrían requerir modificaciones en la arquitectura, implementación o planificación del proyecto.

3. Requisitos de interfaz externa

3.1 Interfaces de usuario

La interfaz de usuario constituye el principal medio de interacción entre el sistema y los usuarios de la simulación. El diseño deberá seguir principios de claridad, consistencia y facilidad de uso, permitiendo que los operadores puedan supervisar y controlar la misión de forma eficiente.

La ventana principal estará dividida en cuatro áreas funcionales principales:

1. **Vista Orbital:** *mostrará una representación gráfica bidimensional de la Tierra, la Luna y la nave espacial, incluyendo la trayectoria calculada durante la simulación.*
2. **Panel de Telemetría:** *presentará información actualizada sobre los parámetros más relevantes de la misión, incluyendo velocidad orbital, altitud respecto a la Luna y tiempo transcurrido de misión (MET).*
3. **Panel de Control:** *contendrá los controles necesarios para iniciar, pausar y reiniciar la simulación.*
4. **Consola de Alertas:** *mostrará mensajes de advertencia y eventos críticos detectados durante la ejecución de la misión.*

La interfaz deberá actualizarse dinámicamente conforme se generen nuevos datos orbitales. Todos los elementos visuales deberán mantener una distribución consistente y permanecer visibles durante la simulación.

Los botones principales incluirán:

- *Iniciar Simulación*
- *Pausar Simulación*
- *Reiniciar Simulación*
- *Salir*

Los mensajes de error deberán mostrarse mediante cuadros de diálogo o notificaciones visibles que describan claramente la causa del problema y, cuando sea posible, las acciones recomendadas para solucionarlo.

Las alertas operativas utilizarán un sistema de colores estandarizado:

- Verde: operación normal.
- Naranja: advertencia.
- Rojo: condición crítica.

Los componentes que requieren interfaz de usuario incluyen el módulo de visualización orbital, el panel de telemetría, el sistema de control de simulación y la consola de alertas.

Los detalles específicos del diseño gráfico, distribución visual, tamaños de componentes y diagramas de pantalla serán documentados en una futura Especificación de Interfaz de Usuario (UI Specification).

3.2 Interfaces de hardware

El sistema interactuará con componentes de hardware estándar disponibles en equipos personales utilizados en entornos académicos.

Teclado

El teclado permitirá la activación de funciones de control y navegación dentro de la aplicación. Podrán incorporarse atajos de teclado en versiones futuras para acciones frecuentes como iniciar, pausar o reiniciar la simulación.

Ratón

El ratón permitirá seleccionar controles gráficos, interactuar con botones de comando y navegar por la interfaz visual.

Monitor

El monitor será utilizado para representar toda la información gráfica generada por el sistema. La aplicación deberá visualizar correctamente la interfaz en resoluciones iguales o superiores a 1366 × 768 píxeles.

Procesador y Memoria

El sistema utilizará los recursos de procesamiento del equipo para ejecutar simultáneamente los cálculos orbitales y la actualización gráfica. Se recomienda un procesador de doble núcleo y al menos 4 GB de memoria RAM para garantizar un funcionamiento adecuado.

Interacción Hardware-Software

La entrada de datos será recibida desde teclado y ratón mediante los mecanismos estándar proporcionados por el sistema operativo y JavaFX. La salida de datos consistirá principalmente en información visual mostrada en pantalla.

No se requiere comunicación con dispositivos externos especializados ni sensores físicos durante la primera versión del proyecto.

3.3 Interfaces de software

El sistema se apoyará en varios componentes de software externos que proporcionan servicios esenciales para su funcionamiento.

Orekit

Versión: 13.1.6

Orekit será utilizado como motor principal para los cálculos de mecánica orbital y propagación de trayectorias espaciales.

Datos de entrada:

- *Parámetros orbitales iniciales.*
- *Tiempo de simulación.*
- *Configuración de la misión.*

Datos de salida:

- *Posición de la nave.*
- *Velocidad orbital.*
- *Información temporal.*
- *Datos de referencia orbital.*

Estos datos serán consumidos por los módulos de visualización, telemetría y alertas.

JavaFX

Versión: JavaFX 17 o superior.

JavaFX proporcionará la infraestructura gráfica necesaria para construir la interfaz de usuario.

Servicios utilizados:

- *Renderizado gráfico.*
- *Gestión de eventos.*
- *Controles visuales.*
- *Actualización dinámica de componentes.*

Java Virtual Machine (JVM)

Versión mínima: Java 17.

La JVM proporcionará el entorno de ejecución necesario para todos los componentes del sistema.

Servicios utilizados:

- *Gestión de memoria.*
- *Ejecución multihilo.*
- *Manejo de excepciones.*
- *Administración de recursos del sistema.*

Git y GitHub

Estas herramientas serán utilizadas para la gestión del código fuente y el control de versiones durante el desarrollo del proyecto.

Intercambio de Datos Interno

La arquitectura del sistema utilizará una comunicación desacoplada entre módulos.

El motor orbital generará datos de estado que serán compartidos con:

- *Módulo de visualización.*
- *Módulo de telemetría.*
- *Sistema de alertas.*

La comunicación entre componentes deberá realizarse mediante objetos de estado compartidos o mecanismos seguros de sincronización compatibles con la arquitectura multihilo de Java.

Como restricción de implementación, los cálculos realizados por Orekit deberán ejecutarse en un hilo independiente al utilizado por JavaFX para evitar bloqueos en la interfaz gráfica.

3.4 Interfaces de comunicaciones

La primera versión del sistema funcionará completamente de forma local y no requerirá comunicación con servicios externos.

No se utilizarán protocolos de red como HTTP, HTTPS, FTP, SMTP o WebSocket para la operación normal del simulador.

Toda la información generada durante la ejecución permanecerá dentro del entorno local de la aplicación.

En futuras versiones podrían incorporarse mecanismos de comunicación para:

- *Sincronización de simulaciones.*
- *Compartición de telemetría.*
- *Monitoreo remoto.*
- *Integración con servicios web científicos.*
- *Actualización automática de parámetros orbitales.*

En caso de implementarse comunicaciones externas en versiones futuras, deberán utilizarse protocolos seguros basados en HTTPS y mecanismos de cifrado compatibles con los estándares actuales de seguridad informática.

Asimismo, cualquier intercambio de información deberá incorporar mecanismos de validación y sincronización que garanticen la integridad de los datos transmitidos.

4. Características del sistema

4.1 Propagación Orbital

4.1.1 Descripción y prioridad

Esta característica permite calcular y actualizar la trayectoria orbital de la nave espacial durante toda la simulación utilizando la biblioteca Orekit. Constituye el núcleo funcional del sistema, ya que proporciona los datos físicos necesarios para la visualización, telemetría y generación de alertas.

Prioridad: Alta

Beneficio: 9/9

Penalización por ausencia: 9/9

Costo de implementación: 8/9

Riesgo técnico: 7/9

4.1.2 Secuencias de estímulo/respuesta

- 1. El usuario inicia la simulación.*
- 2. El sistema carga los parámetros orbitales iniciales.*
- 3. El motor Orekit comienza la propagación orbital.*
- 4. El sistema calcula la posición y velocidad de la nave.*
- 5. Los datos calculados son enviados a los módulos de visualización y telemetría.*
- 6. El proceso continúa hasta que el usuario pause o reinicie la simulación.*

En caso de error en los parámetros orbitales, el sistema mostrará un mensaje descriptivo y detendrá el inicio de la simulación.

4.1.3 Requisitos funcionales

REQ-RF1-01: *El sistema deberá calcular la trayectoria translunar utilizando la biblioteca Orekit.*

REQ-RF1-02: *El sistema deberá actualizar continuamente la posición orbital de la nave durante la simulación.*

REQ-RF1-03: *El sistema deberá recalcular el estado orbital cuando el usuario reinicie la simulación.*

REQ-RF1-04: *El sistema deberá mantener la coherencia temporal de los cálculos durante toda la misión.*

REQ-RF1-05: *El sistema deberá validar los parámetros orbitales antes de iniciar cualquier cálculo.*

REQ-RF1-06: *Si ocurre un error durante la propagación orbital, el sistema deberá notificarlo al usuario y registrar el evento.*

4.2 Visualización Gráfica

4.2.1 Descripción y prioridad

Esta característica proporciona una representación visual de la misión espacial, permitiendo observar la posición relativa de la Tierra, la Luna y la nave durante la simulación.

Prioridad: *Alta*

Beneficio: 8/9

Penalización por ausencia: 7/9

Costo de implementación: 6/9

Riesgo técnico: 5/9

4.2.2 Secuencias de estímulo/respuesta

- 1. El usuario inicia la simulación.*
- 2. El sistema recibe los datos orbitales calculados.*
- 3. La interfaz gráfica actualiza las posiciones de los objetos representados.*
- 4. El usuario observa la evolución de la trayectoria.*
- 5. La representación continúa actualizándose hasta finalizar la simulación.*

4.2.3 Requisitos funcionales

REQ-RF2-01: *El sistema deberá mostrar gráficamente la Tierra, la Luna y la nave espacial.*

REQ-RF2-02: *El sistema deberá actualizar la representación visual en tiempo real.*

REQ-RF2-03: *El sistema deberá reflejar los cambios de posición calculados por el motor orbital.*

REQ-RF2-04: *El sistema deberá mantener una escala visual coherente durante la simulación.*

REQ-RF2-05: *El sistema deberá mostrar una notificación visual cuando no existan datos válidos para representar.*

4.3 Telemetría

4.3.1 Descripción y prioridad

La característica de telemetría permite monitorear continuamente los parámetros más importantes

de la misión, facilitando la supervisión del estado actual de la nave.

Prioridad: Alta

Beneficio: 8/9

Penalización por ausencia: 8/9

Costo de implementación: 4/9

Riesgo técnico: 3/9

4.3.2 Secuencias de estímulo/respuesta

1. *El motor orbital calcula un nuevo estado de la nave.*
2. *El sistema obtiene los datos relevantes.*
3. *Los datos son enviados al panel de telemetría.*
4. *La interfaz actualiza los valores mostrados al usuario.*
5. *El usuario monitorea el progreso de la misión.*

4.3.3 Requisitos funcionales

REQ-RF3-01: *El sistema deberá mostrar la velocidad orbital actual de la nave.*

REQ-RF3-02: *El sistema deberá mostrar la altitud respecto a la superficie lunar.*

REQ-RF3-03: *El sistema deberá mostrar el tiempo transcurrido de misión (MET).*

REQ-RF3-04: *El sistema deberá actualizar los datos de telemetría al menos una vez por segundo.*

REQ-RF3-05: *El sistema deberá indicar claramente cuando un dato no pueda ser calculado.*

REQ-RF3-06: *El sistema deberá mantener la sincronización entre los datos mostrados y los cálculos orbitales.*

4.4 Control de Simulación

4.4.1 Descripción y prioridad

Esta característica proporciona los controles necesarios para administrar el ciclo de ejecución de la simulación, permitiendo al usuario iniciar, pausar y reiniciar la misión.

Prioridad: Alta

Beneficio: 9/9

Penalización por ausencia: 9/9

Costo de implementación: 3/9

Riesgo técnico: 2/9

4.4.2 Secuencias de estímulo/respuesta

1. *El usuario presiona el botón Iniciar.*
2. *El sistema comienza la simulación.*
3. *El usuario presiona el botón Pausar.*
4. *El sistema detiene temporalmente la propagación orbital.*

5. *El usuario presiona el botón Reiniciar.*
6. *El sistema restaura las condiciones iniciales y reinicia la simulación.*

4.4.3 Requisitos funcionales

REQ-RF4-01: *El sistema deberá proporcionar un control para iniciar la simulación.*

REQ-RF4-02: *El sistema deberá proporcionar un control para pausar la simulación.*

REQ-RF4-03: *El sistema deberá proporcionar un control para reiniciar la simulación.*

REQ-RF4-04: *El sistema deberá restaurar los parámetros iniciales cuando se ejecute un reinicio.*

REQ-RF4-05: *El sistema deberá impedir la ejecución simultánea de múltiples simulaciones.*

REQ-RF4-06: *El sistema deberá informar al usuario sobre el estado actual de la simulación.*

4.5 Sistema de Alertas

4.5.1 Descripción y prioridad

Esta característica supervisa continuamente las condiciones de la misión y notifica al usuario cuando se detectan desviaciones o eventos críticos que requieran atención.

Prioridad: *Media*

Beneficio: *7/9*

Penalización por ausencia: *6/9*

Costo de implementación: *4/9*

Riesgo técnico: *4/9*

4.5.2 Secuencias de estímulo/respuesta

1. *El motor orbital detecta una condición anómala.*
2. *El sistema evalúa el nivel de riesgo.*
3. *La consola de alertas cambia de estado.*
4. *El usuario recibe la notificación visual correspondiente.*

4.5.3 Requisitos funcionales

REQ-RF5-01: *El sistema deberá mostrar alertas visuales cuando se detecten condiciones anómalas.*

REQ-RF5-02: *El sistema deberá cambiar el color de la consola a naranja para advertencias.*

REQ-RF5-03: *El sistema deberá cambiar el color de la consola a rojo para situaciones críticas.*

REQ-RF5-04: *El sistema deberá registrar las alertas generadas durante la simulación.*

REQ-RF5-05: *El sistema deberá mostrar una descripción breve de la condición detectada.*

REQ-RF5-06: El sistema deberá eliminar automáticamente las alertas cuando la condición haya sido corregida o finalizada.

5. Otros requisitos no funcionales

5.1 Requisitos de desempeño

El rendimiento del sistema es un factor crítico para garantizar una experiencia de simulación fluida y una representación precisa de los eventos orbitales. El simulador deberá mantener una tasa mínima y estable de 30 cuadros por segundo (FPS) durante la ejecución normal de la misión, evitando interrupciones visuales, retrasos perceptibles o congelamientos de la interfaz gráfica.

La telemetría mostrada al usuario deberá actualizarse en intervalos no mayores a un segundo, asegurando que la información presentada refleje adecuadamente el estado actual de la nave espacial. Los datos de velocidad, altitud, posición y tiempo de misión deberán mantenerse sincronizados con los cálculos realizados por el motor de simulación.

Las acciones iniciadas por el usuario, incluyendo iniciar, pausar y reiniciar la simulación, deberán generar una respuesta visible en un tiempo inferior a 500 milisegundos. Asimismo, el sistema deberá ser capaz de ejecutar simultáneamente los cálculos orbitales y la actualización gráfica sin degradar significativamente el desempeño general de la aplicación.

La arquitectura deberá permitir que el motor de propagación orbital y la interfaz gráfica operen de manera concurrente, garantizando que los cálculos complejos no afecten la capacidad de interacción del usuario.

5.2 Requisitos de seguridad

Aunque el sistema tiene un propósito académico y no controla hardware real, deberá implementar mecanismos que garanticen la estabilidad y confiabilidad de la simulación. La aplicación no deberá finalizar inesperadamente durante una misión activa debido a errores controlables o entradas inválidas del usuario.

El sistema deberá detectar excepciones producidas por fallos de cálculo, errores de configuración o datos faltantes y gestionarlos adecuadamente mediante mensajes informativos en lugar de detener la ejecución abruptamente. Cuando ocurra una condición de error recuperable, la simulación deberá mantener el último estado válido conocido o regresar a una configuración segura previamente definida.

Las operaciones críticas relacionadas con el cálculo orbital deberán ejecutarse de manera controlada para evitar inconsistencias en los datos de telemetría. Además, el sistema deberá registrar los errores relevantes para facilitar actividades de depuración, mantenimiento y mejora continua.

La arquitectura multihilo deberá minimizar riesgos asociados a bloqueos, condiciones de carrera o pérdida de sincronización entre el motor físico y la interfaz gráfica.

5.3 Requisitos de seguridad

El sistema no recopilará, almacenará ni transmitirá información personal de los usuarios. Toda la funcionalidad del simulador deberá operar localmente sin requerir la creación de cuentas ni la autenticación de usuarios en servicios externos.

Los parámetros ingresados por el usuario deberán validarse antes de ser utilizados por el motor de simulación. Cualquier valor fuera de los límites permitidos deberá ser rechazado y acompañado por una notificación clara que indique la naturaleza del error detectado.

La aplicación deberá evitar que entradas incorrectas produzcan resultados inconsistentes o afecten la estabilidad del sistema. Asimismo, deberá garantizar que los datos de configuración de la simulación permanezcan íntegros durante toda la ejecución.

En futuras versiones que incorporen almacenamiento de configuraciones o intercambio de datos, deberán implementarse mecanismos adicionales de protección y control de acceso acordes con las mejores prácticas de desarrollo seguro.

5.4 Atributos de calidad del software

El sistema deberá diseñarse priorizando la usabilidad, permitiendo que los usuarios comprendan y utilicen sus funciones principales con una capacitación mínima. La interfaz gráfica deberá presentar información clara, consistente y fácilmente interpretable incluso para usuarios con experiencia limitada en simulación espacial.

La mantenibilidad será un objetivo fundamental del proyecto. El código deberá estar organizado en módulos claramente definidos, documentado adecuadamente y estructurado siguiendo principios de programación orientada a objetos para facilitar futuras modificaciones y correcciones.

La portabilidad deberá garantizar que la aplicación pueda ejecutarse sin modificaciones significativas en sistemas operativos Windows, Linux y macOS siempre que dispongan de un entorno Java compatible.

La escalabilidad permitirá incorporar nuevas funcionalidades, misiones espaciales, cuerpos celestes o modelos físicos más avanzados sin requerir una reestructuración completa de la arquitectura existente.

La robustez deberá asegurar que el sistema continúe operando correctamente ante errores previsibles, entradas inválidas o situaciones excepcionales. Del mismo modo, la fiabilidad deberá garantizar que los resultados producidos por la simulación sean consistentes y reproducibles bajo las mismas condiciones de ejecución.

Finalmente, la interoperabilidad deberá facilitar la integración futura con bibliotecas científicas adicionales, motores de visualización avanzados y herramientas complementarias utilizadas en simulación aeroespacial.

5.5 Reglas de negocio

La simulación representará una única misión activa por instancia del sistema. No se permitirá la ejecución simultánea de múltiples simulaciones dentro de la misma sesión de trabajo con el fin de preservar la coherencia de los datos y simplificar la administración de recursos computacionales.

Toda trayectoria orbital deberá ser calculada utilizando la biblioteca Orekit como fuente principal para la propagación orbital y la gestión de referencias temporales. No se permitirá sustituir estos cálculos por aproximaciones simplificadas que comprometan los objetivos educativos del proyecto.

Cada simulación deberá comenzar desde un conjunto de condiciones iniciales previamente definidas por el equipo de desarrollo, garantizando la repetibilidad de los experimentos y la comparación consistente de resultados.

Los eventos críticos detectados durante la misión deberán generar alertas visuales automáticas para informar al usuario sobre desviaciones significativas de la trayectoria esperada. Estas alertas formarán parte del proceso normal de monitoreo y apoyo a la toma de decisiones durante la simulación.

Todas las modificaciones futuras realizadas al sistema deberán documentarse en el repositorio del proyecto y reflejarse en el archivo decisiones.md para mantener la trazabilidad técnica y académica del desarrollo.

6. Otros requisitos

El diseño arquitectónico del sistema deberá facilitar futuras ampliaciones sin requerir modificaciones significativas en los componentes principales existentes. La solución deberá prepararse para incorporar nuevas misiones inspiradas en el programa Artemis, incluyendo diferentes perfiles de vuelo, trayectorias orbitales y escenarios operativos.

La arquitectura deberá permitir la incorporación futura de capacidades de visualización tridimensional utilizando motores gráficos especializados, manteniendo la compatibilidad con la lógica de simulación desarrollada inicialmente. Asimismo, deberá contemplar la posibilidad de integrar modelos físicos más complejos relacionados con maniobras orbitales, correcciones de trayectoria y eventos de asistencia gravitacional.

El sistema también deberá estar preparado para soportar nuevas fuentes de datos científicas, simulaciones de múltiples vehículos espaciales y mecanismos avanzados de análisis de

telemetría. Estas extensiones deberán poder implementarse mediante módulos adicionales sin afectar significativamente los componentes existentes.

Como parte de los objetivos académicos del proyecto, el software deberá promover buenas prácticas de ingeniería de software, incluyendo modularidad, reutilización de componentes, documentación continua y control de versiones. Esto permitirá que futuras generaciones de estudiantes puedan utilizar el proyecto como base para nuevas investigaciones, mejoras y desarrollos relacionados con la exploración espacial.

Apéndice A: Glosario

Orekit

Biblioteca de código abierto desarrollada en Java especializada en dinámica espacial y mecánica orbital. Proporciona herramientas para propagación de órbitas, manejo de sistemas de referencia, modelos gravitacionales, gestión del tiempo espacial y cálculos utilizados en aplicaciones aeroespaciales. En este proyecto constituye el motor principal para el cálculo de trayectorias orbitales y la generación de datos de telemetría.

Artemis II

Segunda misión tripulada del programa Artemis de la NASA. Su objetivo es realizar un sobrevuelo lunar con astronautas a bordo de la nave Orion antes de futuras misiones de alunizaje. La misión sirve como principal referencia conceptual para el desarrollo del simulador Oasis Lunar.

Telemetría

Conjunto de datos operativos generados por un sistema y transmitidos para su monitoreo y análisis. En el contexto de este proyecto incluye información como posición orbital, velocidad, altitud, tiempo transcurrido de misión y estados operativos relevantes.

MET (Mission Elapsed Time)

Tiempo transcurrido desde el inicio oficial de una misión. Se utiliza como referencia temporal principal para coordinar eventos, maniobras y cálculos durante la simulación.

JVM (Java Virtual Machine)

Entorno de ejecución que permite ejecutar aplicaciones Java de manera independiente del sistema operativo. La JVM administra la memoria, la ejecución de procesos, la gestión de excepciones y otros servicios fundamentales requeridos por el simulador.

JavaFX

Framework gráfico utilizado para desarrollar interfaces de usuario en aplicaciones Java. Proporciona herramientas para crear ventanas, controles visuales, animaciones, gráficos y eventos de interacción utilizados por la interfaz del simulador.

LEO (Low Earth Orbit)

Órbita terrestre baja ubicada aproximadamente entre 160 y 2,000 kilómetros sobre la superficie terrestre. Es una de las órbitas más utilizadas para satélites y suele representar la fase inicial de muchas misiones espaciales.

FPS (Frames Per Second)

Unidad utilizada para medir la cantidad de cuadros o imágenes que se muestran por segundo en una aplicación gráfica. Un valor elevado de FPS contribuye a una experiencia visual más fluida y realista.

Propagación Orbital

Proceso matemático mediante el cual se calcula la evolución futura de una órbita a partir de un conjunto de condiciones iniciales y modelos físicos determinados.

Trayectoria Translunar

Ruta seguida por una nave espacial durante su desplazamiento desde la órbita terrestre hacia las proximidades de la Luna.

Simulación

Representación computacional de un sistema o fenómeno real con el propósito de analizar su comportamiento bajo determinadas condiciones.

Arquitectura Multihilo

Modelo de ejecución que permite realizar múltiples tareas de manera concurrente utilizando varios hilos de procesamiento. En este proyecto permite separar los cálculos orbitales de la actualización de la interfaz gráfica.

CAPCOM

Siglas de Capsule Communicator. En las misiones espaciales representa al responsable de la comunicación y supervisión operativa entre la misión y el centro de control. Dentro del proyecto corresponde a un rol encargado de monitorear alertas y eventos.

SRS (Software Requirements Specification)

Documento formal que describe de manera completa y verificable los requisitos funcionales y no funcionales de un sistema de software.

Apéndice C: Lista por determinar

Los siguientes elementos aún no han sido definidos completamente y deberán ser revisados y actualizados en futuras versiones del proyecto.

TBD-01: Parámetros Orbitales Definitivos de la Misión

Se encuentran pendientes los valores exactos que definirán las condiciones iniciales de la simulación, incluyendo elementos orbitales, tiempos de referencia, altitudes iniciales y configuraciones específicas de la trayectoria translunar.

Impacto:

Puede afectar los cálculos orbitales, la telemetría y la validación de resultados.

Responsable:

Equipo de Arquitectura y Simulación.

TBD-02: Diseño Final de la Interfaz Gráfica

Aún no se ha definido completamente la distribución visual de la aplicación, los esquemas de colores, los componentes gráficos definitivos y la organización de las ventanas y paneles.

Impacto:

Puede afectar la experiencia de usuario y la implementación de la interfaz JavaFX.

Responsable:

Equipo de Desarrollo e Interfaz de Usuario.

TBD-03: Catálogo Completo de Alertas

Se encuentra pendiente la definición detallada de todas las condiciones operativas que generarán advertencias o alertas críticas durante la simulación.

Ejemplos preliminares:

- *Desviación significativa de trayectoria.*
- *Error en cálculo orbital.*
- *Pérdida de sincronización temporal.*
- *Valores fuera de rango en telemetría.*

Impacto:

Puede afectar el diseño del sistema de monitoreo y notificaciones.

Responsable:

Equipo de Requisitos y Simulación.

TBD-04: Escenarios Avanzados de Simulación

La versión inicial contempla únicamente una simulación básica inspirada en Artemis II. Futuras iteraciones podrán incluir escenarios más complejos.

Posibles escenarios:

- *Maniobras de corrección de trayectoria.*
- *Simulación de fallos operativos.*
- *Variaciones en parámetros orbitales.*
- *Misiones Artemis posteriores.*
- *Simulación de múltiples vehículos espaciales.*

Impacto:

Influye en la escalabilidad de la arquitectura y en la planificación de futuras versiones.

Responsable:

Equipo completo del proyecto.

TBD-05: Métricas Finales de Desempeño

Se encuentran pendientes los valores definitivos de consumo de memoria, utilización de CPU, tiempos de respuesta y métricas de rendimiento obtenidas durante las pruebas del sistema.

TBD-06: Estrategia de Validación y Pruebas

Se deberá definir el conjunto completo de casos de prueba, criterios de aceptación y procedimientos de validación que permitirán verificar el cumplimiento de todos los requisitos establecidos en este SRS.

TBD-07: Versión Definitiva de Dependencias Externas

Se deberá confirmar la versión específica de Orekit, JavaFX y demás bibliotecas que serán utilizadas en la entrega final del proyecto para garantizar la reproducibilidad del entorno de desarrollo.